Miniprojekt 1 – Regresja liniowa

Metody probabilistyczne w uczeniu maszynowym

Mateusz Wojaczek

Streszczenie

W tym raporcie opisuję implementację regresji liniowej na danych z pliku `dane.data` – 8 cech wejściowych i jedna zmienna objaśniana. Zacząłem od prostego modelu liniowego, potem testowałem różne funkcje bazowe (wielomiany, funkcje gaussowskie, Fouriera, sigmoidalne), regularyzacje L1, L2 i Elastic Net. Całość zaimplementowałem od zera w NumPy, bez gotowych bibliotek ML.

1 Przygotowanie danych

1.1 Wczytywanie i podział

Dane wczytuję z pliku `dane.data`. Przy każdym uruchomieniu losuję kolejność obserwacji, a następnie dzielę je na trzy zbiory w proporcji 0.6 : 0.2 : 0.2 (treningowy, walidacyjny, testowy). Hiperparametry dobieram na zbiorze walidacyjnym, a ostateczną ocenę modelu robię na zbiorze testowym.

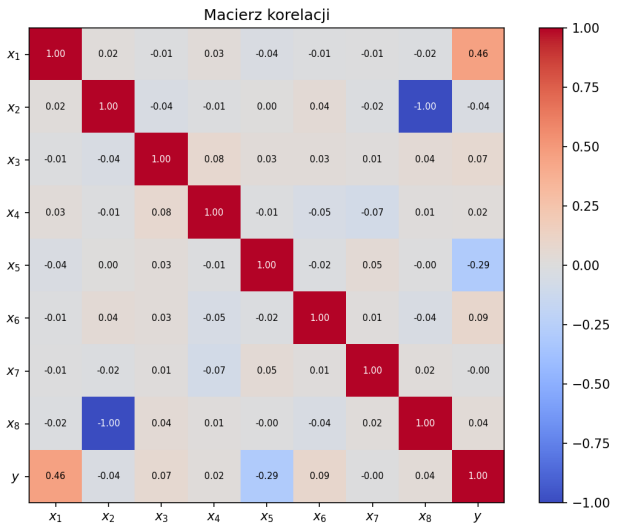
1.2 Standaryzacja

Przed trenowaniem standaryzuję cechy wejściowe – od każdej cechy odejmuję jej średnią i dzielę przez odchylenie standardowe (oba liczone na zbiorze treningowym). Zbiory walidacyjny i testowy transformuję tymi samymi parametrami co treningowy. Zmiennej y nie skaluję.

1.3 Analiza korelacji

Na ustandaryzowanych danych policzyłem macierz korelacji Pearsona. Widać z niej kilka rzeczy:

- Cechy x_2 i x_7 są ekstremalnie silnie skorelowane.
- Zmienna y jest najsilniej skorelowana z cechami x_1, x_4 i x_5.
- Pozostałe cechy (x_3, x_6, x_8) mają słabszą korelację z y.



Rysunek 1: Macierz korelacji między cechami a zmienną objaśnianą

2 Model liniowy – rozwiązanie analityczne

Zacząłem od najprostszego modelu: regresja liniowa z funkcjami bazowymi $\varphi(x) = [1, x_1, \ldots, x_8]$, kwadratowa funkcja straty:

$$\ell(\theta) = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$$

Rozwiązanie analityczne to $\theta = (X^T X + \lambda I)^{-1} X^T y$, gdzie I ma wyzerowany element $(0, 0)$ żeby nie regularyzować biasu. Dla $\lambda \to 0$ dostałem:

Train MSE: 10602.69 Test MSE: 11440.03

2.1 Wpływ regularyzacji L2 na rozwiązanie analityczne

Sprawdziłem jak zmienia się MSE przy różnych wartościach λ w regularyzacji grzbietowej:

λ	Train MSE	Test MSE
10^{-6}	10602.65	11436.51
0.1	10602.65	11436.55
0.5	10602.65	11436.69
1	10602.66	11436.86
5	10602.73	11438.32
10	10602.95	11440.26
50	10609.61	11460.23
100	10628.40	11494.63
1000	11514.92	12546.28

Tabela 1: MSE rozwiązania analitycznego dla różnych λ (regularyzacja L2)

Widać, że przy małych λ regularyzacja praktycznie nie zmienia wyniku, a przy dużych wartościach model jest za bardzo ograniczony i MSE rośnie. W przypadku prostego modelu liniowego regularyzacja nie daje poprawy.

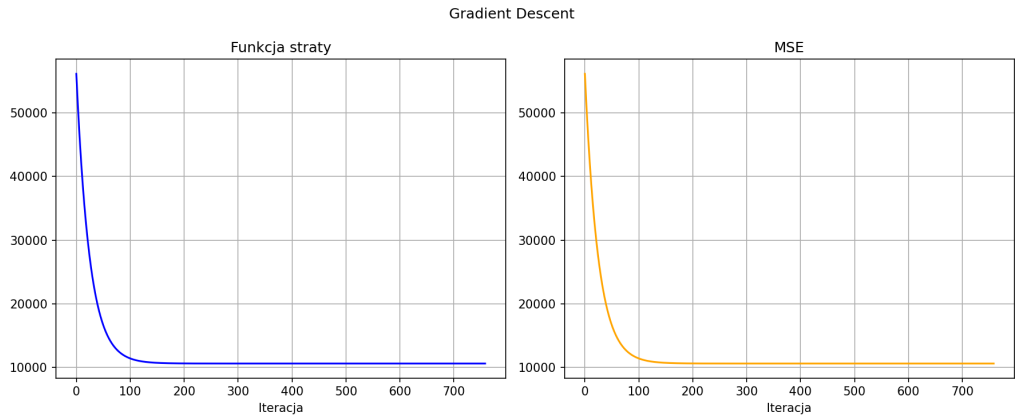
3 Metody gradientowe

3.1 Gradient descent (pełny)

Zaimplementowałem klasyczny gradient descent z learning rate $\eta = 0.01$. Warunek stopu: albo ustalone k iteracji, albo norma różnicy θ poniżej progu 10^{-6}. Gradient kwadratowej funkcji straty:

$$\nabla \ell(\theta) = \frac{2}{m} X^T (X\theta - y)$$

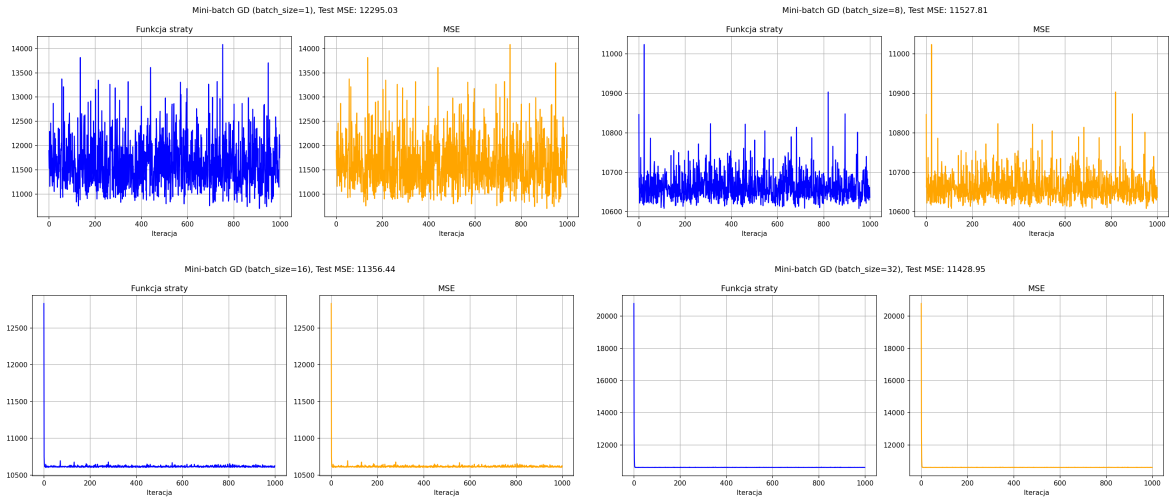
Przy 1000 iteracjach gradient ładnie zbiega do zera, a MSE stabilizuje się.



Rysunek 2: Gradient i MSE w kolejnych iteracjach – pełny gradient descent

3.2 Mini-batch gradient descent

Zaimplementowałem też wersję mini-batch. Przetestowałem różne rozmiary batchy: 1, 8, 16, 32.



Rysunek 3: Mini-batch gradient descent dla różnych rozmiarów batcha

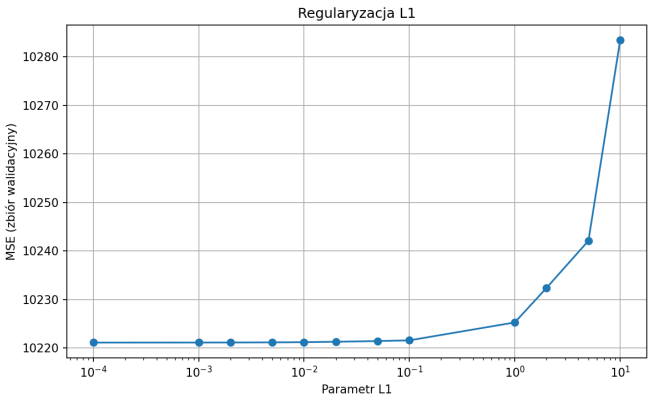
Przy mini-batchu gradient nie zbiega gładko do zera, tylko oscyluje – to normalne, bo w każdej iteracji liczymy gradient na losowym podzbiorze danych. Mimo to model uczy się poprawnie, a cały proces jest znacznie szybszy niż pełny GD.

4 Regularyzacja – dobór hiperparametrów

Do wyznaczenia parametrów regularyzacji użyłem zbioru walidacyjnego. Testowałem wartości $\lambda \in \{0, 0.0001, 0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1, 1, 2, 5, 10\}$.

4.1 Regularyzacja L1 (Lasso)

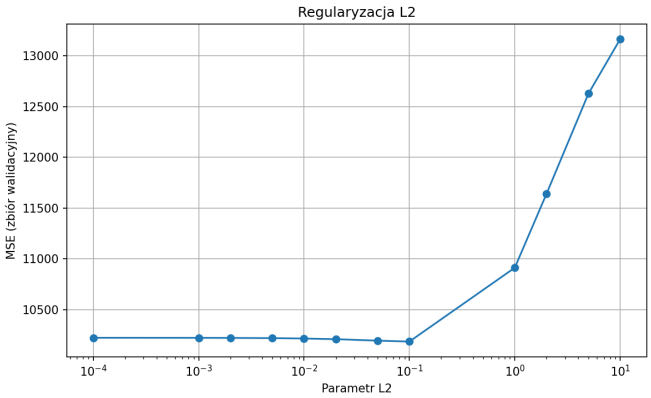
Do gradientu dodaję składnik $\lambda_1 \cdot \text{sign}(\theta)$ (z pominięciem biasu).



Rysunek 4: MSE na zbiorze walidacyjnym dla różnych parametrów L1

4.2 Regularyzacja L2 (Ridge)

Do gradientu dodaję składnik $2\lambda_2 \cdot \theta$ (bez biasu).



Rysunek 5: MSE na zbiorze walidacyjnym dla różnych parametrów L2

4.3 Elastic Net (L1 + L2)

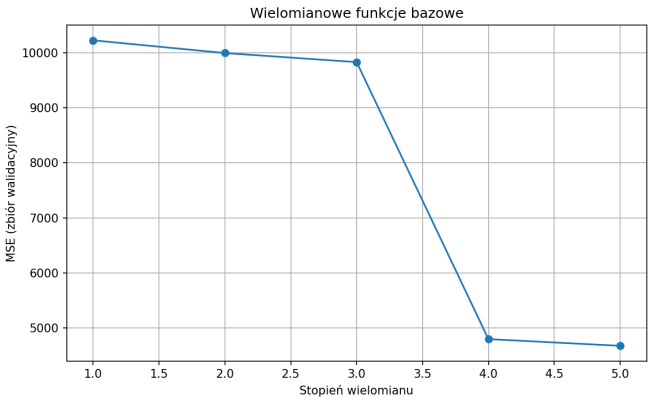
Przetestowałem kombinacje L1 i L2 – pełen grid $\lambda_1 \times \lambda_2$. Najlepsze parametry to $\lambda_1 = 0$, $\lambda_2 = 0.1$ z MSE $= 10183.48$ na zbiorze walidacyjnym. W przypadku prostego modelu liniowego regularyzacja nie przynosi dużej poprawy.

5 Funkcje bazowe

Tu zaczyna się najciekawsza część. Zamiast modelować y jako kombinację liniową surowych cech, transformuję cechy przez funkcje bazowe i dopiero na wyniku robię regresję liniową.

5.1 Wielomiany

Klasa `Polynomial(degree)` generuje cechy $[1, x_1, \ldots, x_8, x_1^2, \ldots, x_8^2, \ldots, x_1^d, \ldots, x_8^d]$. Testowałem stopnie od 1 do 5.

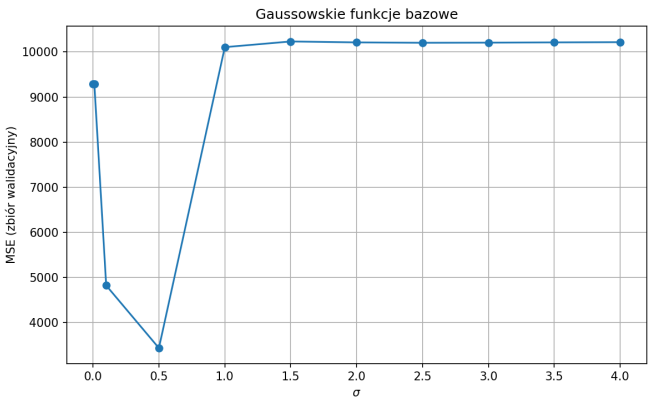


Rysunek 6: MSE na zbiorze walidacyjnym dla różnych stopni wielomianu

Najlepszy wynik dał wielomian stopnia 5 – Test MSE: **4942.57**. To ogromna poprawa w stosunku do modelu liniowego (11440).

5.2 Funkcje gaussowskie

Dla każdej cechy generuję funkcje postaci $\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$, gdzie μ to centra rozłożone równomiernie w zakresie danych. Testowałem różne wartości σ.

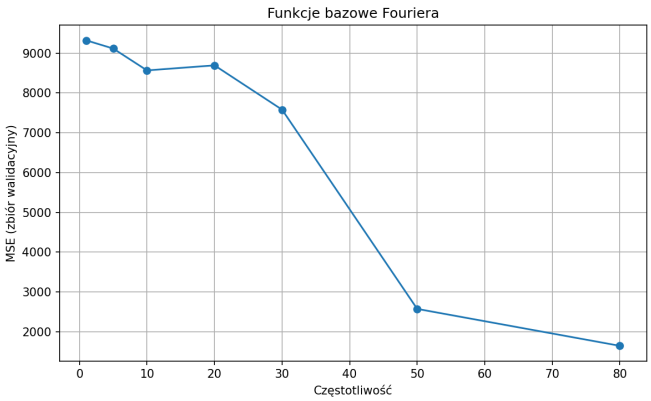


Rysunek 7: MSE na zbiorze walidacyjnym dla różnych σ funkcji gaussowskiej

Najlepsza gaussowska (połączona z cechami liniowymi, $\sigma = 0.5$) dała Test MSE: **4042.12**.

5.3 Funkcje Fouriera

Generuję cechy $[\sin(2\pi kx), \cos(2\pi kx)]$ dla $k = 1, \ldots, n$. Testowałem różne częstotliwości.

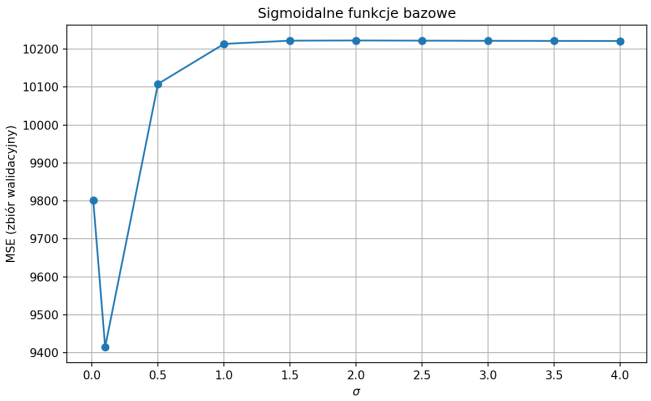


Rysunek 8: MSE na zbiorze walidacyjnym dla różnych częstotliwości Fouriera

Fourier okazał się najlepszą funkcją bazową – Test MSE: **1621.60**.

5.4 Funkcje sigmoidalne

Testowałem też sigmoidy $\frac{1}{1+e^{-(x-\mu)/\sigma}}$ z różnymi parametrami σ.



Rysunek 9: MSE na zbiorze walidacyjnym dla różnych σ sigmoidy

Sigmoid wypadł najsłabiej – Test MSE: **10389.92**. Najwyraźniej sigmoidy nie dodają wiele ponad zwykłe cechy liniowe dla tych danych.

5.5 Porównanie funkcji bazowych

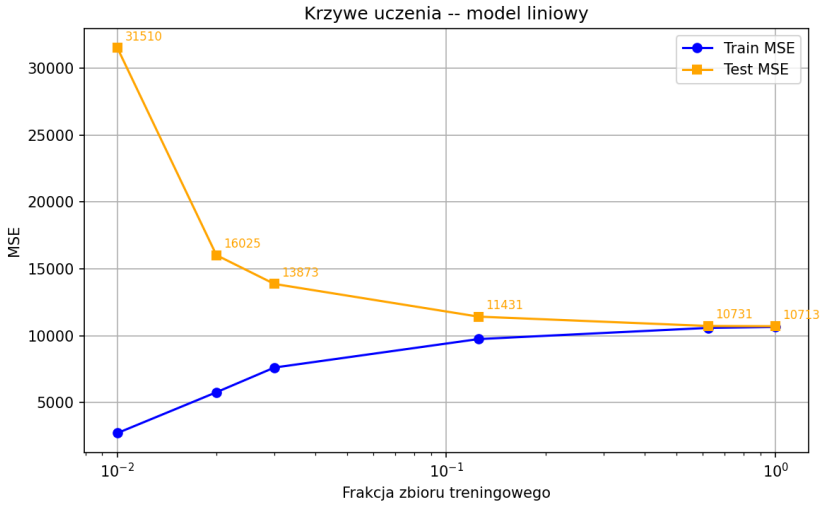
Funkcja bazowa	Test MSE
Liniowa	11440.03
Wielomian (st. 5)	4942.57
Gaussian ($\sigma = 0.5$)	4042.12
Fourier (freq = 80)	**1621.60**
Sigmoid ($\sigma = 0.1$)	10389.92

Tabela 2: Porównanie Test MSE dla różnych funkcji bazowych

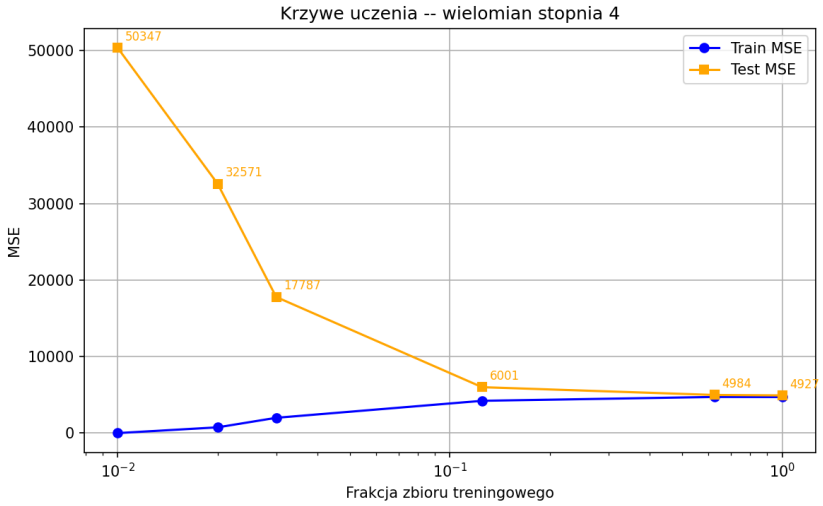
Fourier wygrywa zdecydowanie. Wielomian i gaussowskie też dają dużą poprawę. Sigmoid praktycznie nic nie wnosi.

6 Krzywe uczenia

Sprawdziłem jak model radzi sobie na różnych frakcjach zbioru treningowego: 0.01, 0.02, 0.03, 0.125, 0.625, 1. Wyniki uśredniłem po 10 przebiegach z losowym podziałem danych.



Rysunek 10: Krzywe uczenia dla modelu liniowego



Rysunek 11: Krzywe uczenia dla wielomianu stopnia 4

Przy małej ilości danych model treningowy ma niskie MSE (overfitting), a testowe jest ogromne. Wraz ze wzrostem danych treningowe MSE rośnie, a testowe spada – krzywe zbiegają się. Model wielomianowy przy pełnym zbiorze treningowym osiąga znacznie lepsze wyniki niż liniowy.

7 Implementacja

Cały projekt zaimplementowałem modularnie w Pythonie:

- `src/data.py` – wczytywanie danych i podział na zbiory
- `src/scaler.py` – standaryzacja (StandardScaler)

- `src/functions.py` – funkcje bazowe (Linear, Polynomial, Gaussian, Fourier, Sigmoid, Combined)
- `src/loss.py` – funkcje straty (kwadratowa, bezwzględna) z regularyzacją
- `src/model.py` – model regresji (analytical, GD, mini-batch)
- `src/plots.py` – wykresy
- `src/utils.py` – krzywe uczenia

Klasa `Combined` pozwala łączyć różne funkcje bazowe, np. `Combined([Linear(), Gaussian(0.5)])` daje cechy liniowe + gaussowskie.

8 Podsumowanie

- Prosty model liniowy daje MSE $\approx$ 11440 – przyzwoite, ale daleko od ideału.
- Regularyzacja L1/L2 przy modelu liniowym niewiele zmienia – model nie overfituje.
- Funkcje bazowe najwięcej poprawiają: wielomiany zbijają MSE do $\sim$4943, gaussowskie do $\sim$4042, a Fourier aż do $\sim$**1622**.
- Mini-batch GD działa porównywalnie z pełnym GD, ale jest znacznie szybszy.

Najlepszy model to regresja z funkcjami bazowymi Fouriera (połączonymi z liniowymi), osiągający Test MSE $\approx$ **1621.60**.